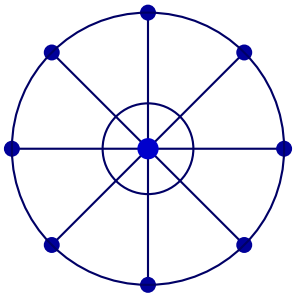
GENESIS



ISLS Genesis Crystal v1.0.0

The ADAMANT Protocol
as Crystallized System Constitution

A Machine-Checkable Foundational Artifact
for Autonomous, Governed, and Traceable Systems

Sebastian Klemm

17 March 2026

Abstract

This document specifies the *Genesis Crystal*: a special Semantic Crystal that encodes the ADAMANT protocol (Open Projection-Phase-Space Systems Framework) as a machine-checkable system constitution. The Genesis Crystal is created at system initialization (`isls init`), stored as the first entry in the crystal archive, and validated at every subsequent run. It contains the architectural axioms, mandatory invariants, conformance class, operator governance rules, gate semantics, trace discipline, and security doctrine of the system — not as documentation, but as verifiable constraints.

If the system's configuration drifts from its constitutional commitments, the Genesis

Crystal validation fails and every report shows the violation. The constitution is not a PDF that can be ignored. It is a crystal that is checked.

No new crate is required. The Genesis Crystal is implemented as extensions to `isls-types` (constitutional constraint types), `isls-harness` (genesis validation), and `isls-cli` (genesis creation and reporting).

Status: Implementation Specification.

Parent: ISLS v1.0.0 + Extensions Phase 1–5.1.

Encodes: ADAMANT Consolidated Master Specification.

Contents

I Foundations 4

1 What Is the Genesis Crystal 4

2 When It Is Created 4

II The Constitutional Constraints 4

3 Mapping ADAMANT to Machine Predicates 5

4 Conformance Classes 7

III Crystal Structure 7

5 Genesis Crystal Schema 7

6 Genesis Crystal Construction 9

IV Runtime Validation 9

7 Genesis Validation 9

8 Genesis in Reports 10

V Constitutional Amendment 11

9 Amendment Crystals 11

VI Implementation 11

10 Changes to Existing Crates 11

10.1 `isls-types` Extensions . . . 11

10.2 `isls-harness` Extensions . . . 12

10.3 `isls-cli` Extensions . . . 13

10.4 `isls-store` Extension . . . 13

VII Acceptance Tests 13

VIII Handoff 14

11 Test Count 14

12 Verification Sequence 15

13 Completion Criteria **15**

Part I
Foundations

1 What Is the Genesis Crystal

Every ISLS instance begins with an empty archive. The first crystal committed to the archive is special: it does not encode a pattern discovered in data or an artifact forged from intent. It encodes the *rules by which the system operates*.

Definition 1.1 (Genesis Crystal)**.** The Genesis Crystal C_0 is a Semantic Crystal whose `constraint_program` field contains the ADAMANT protocol's mandatory invariants encoded as machine-checkable predicates. Its `topology_signature` describes the system's architectural structure at initialization. Its `evidence_chain` proves that the system configuration at initialization satisfies all constitutional requirements.

Definition 1.2 (Constitutional Constraint)**.** A constitutional constraint is a predicate ϕ : SystemState $\to \{0, 1\}$ that encodes one ADAMANT requirement. Each constraint has:

- `id`: unique identifier (e.g., `"ADAMANT-2.0.1"`),
- `axiom`: the ADAMANT axiom or requirement it encodes,
- `predicate`: the machine-checkable condition,
- `severity`: `mandatory` or `recommended`,
- `evidence`: what must be present in the system state for the predicate to be evaluable.

Axiom 1.1 (Constitutional Primacy)**.** The Genesis Crystal is the root of all trust. Every other crystal in the archive is valid only if the Genesis Crystal is valid. If the Genesis Crystal fails validation, the system **MUST** report constitutional violation on every subsequent operation.

Axiom 1.2 (Immutable Constitution)**.** The Genesis Crystal, once committed, **MUST NOT** be modified, replaced, or deleted. Constitutional evolution is handled by *Amendment Crystals* (see section 9), not by mutation.

2 When It Is Created

The Genesis Crystal is created exactly once, at system initialization:

```
isls init [--store sqlite] [--profile standard]
```

The `init` command:

1. Creates the data directory (`~/.isls/`).
2. Initializes the archive, store, and config.
3. Reads the current configuration and registry state.
4. Evaluates all constitutional constraints against the current state.
5. If all mandatory constraints pass: commits the Genesis Crystal.
6. If any mandatory constraint fails: aborts initialization with a diagnostic listing which constraints failed and why.

Part II
The Constitutional Constraints

3 Mapping ADAMANT to Machine Predicates

The following table maps each ADAMANT axiom or requirement to a concrete, machine-checkable predicate on the ISLS system state.

ID	ADAMANT Source	Predicate	Severity
GC-01	Axiom 2.0.1	State domain is bounded: all embeddings normalized to $[0,1]^5$ or explicit normalization declared in config.	mandatory
GC-02	Axiom 2.0.2	Every operator in the registry declares domain, codomain, admissibility predicate, and version.	mandatory
GC-03	Axiom 2.0.3	Trace schema is active: every macro-step produces a TraceEntry with input digest, state digest, gate snapshot.	mandatory
GC-04	Axiom 2.0.4	Extension crates (C12+) do not override kernel invariants I1–I20. Verified by: all 20 AT tests referenced in the registry.	mandatory
GC-05	Axiom 2.0.5	Kernel crates (C1–C11) and extension crates (C12+) are structurally separated: no circular dependencies from extensions back into kernel.	mandatory
GC-06	R-STATE-001	Config declares: dimensionality (5), normalization rule, component partitioning (p, ρ, ω, χ, η), initialization rule, persistence semantics.	mandatory
GC-07	R-COH-002	Coherence functional is bounded $\in [0,1]$, decomposed into at least two components (structural + temporal), and documented in the registry.	mandatory
GC-08	§3.4.4 Gate	Gate function produces outcomes from $\{\text{reject}, \text{hold}, \text{accept}\}$ (or $\{\text{reject}, \text{accept}\}$ for minimal conformance). Gate depends on coherence and at least one additional criterion. Gate is auditable.	mandatory

GC-09	§3.6 Cycle	The canonical control cycle exists: Perceive → Focus → Evaluate → Gate → Act → Trace. Verified by: macro_step implements all phases.	mandatory
GC-10	§5.1 Invariant	All 8 crystal validation checks are active: content_address, dual_consensus, evidence_chain, free_energy, gate_kairos, immutability, operator_versions, por_trace.	mandatory
GC-11	§6.3 Trace	Every committed crystal has a complete evidence chain. No crystal exists without trace.	mandatory
GC-12	§6.6 Replay	Run Descriptor contains sufficient information for deterministic replay: seed, config, registry digests, scheduler config.	mandatory
GC-13	§7.2 Proof	Obligation registry (C12) contains at least the 8 crystal-validation obligations.	mandatory
GC-14	§11.1 Security	Capsule protocol (C14) is available. Secrets are never stored in plaintext in the archive.	mandatory
GC-15	§11.3 Capability	Gateway authentication is configurable. Default: disabled (single-operator). When enabled: bearer token required.	recommended
GC-16	§12.2 Budget	Compute budgets are declared for topology (C16), scale (C18), and PMHD (C21). Budget exhaustion triggers graceful fallback, not crash.	mandatory
GC-17	§13.2 Cross-Layer	No emission bypasses the gate. Verified by: every code path that produces a crystal goes through the gate cascade.	mandatory
GC-18	§14.1 Policy	Master policy exists: operator execution order is deterministic, tie-breaks are declared in RD, no hidden state.	mandatory
GC-19	§16.1 Adaptation	Self-modification (morphogenic controller C8) is bounded: max merges per tick, max parameter drift per epoch.	mandatory

GC-20	§20 Epistemic	"Coherence is not truth" principle: the system does not claim its crystals represent external truth. Crystals represent structural invariance relative to observed data.	mandatory
GC-21	§18 Oversight	Human operator can stop the engine at any time (Ctrl+C, POST /engine/stop). No autonomous action is irreversible without operator confirmation (shadow mode).	mandatory

4 Conformance Classes

Following ADAMANT §5.2, the Genesis Crystal declares a conformance class:

Definition 4.1 (Conformance Classes).

- **C0 — Minimal**: GC-01 through GC-03 satisfied. System can observe and trace.
- **C1 — Governed**: C0 + GC-04 through GC-09. System has operator governance, gate, and cycle.
- **C2 — Auditable**: C1 + GC-10 through GC-13. Full crystal validation, replay, proof obligations.
- **C3 — Secured**: C2 + GC-14 through GC-16. Capsule protocol, authentication, budgets.
- **C4 — Constitutional**: C3 + GC-17 through GC-21. Full ADAMANT compliance including epistemic integrity and human oversight.

The Genesis Crystal records which class the system satisfies at initialization.

Part III
Crystal Structure

5 Genesis Crystal Schema

Genesis Crystal Structure

```
pub struct GenesisCrystal {
    // Standard SemanticCrystal fields
    pub crystal_id: Hash256,
    pub stability_score: f64,        // 1.0 (constitutional)
    pub free_energy: f64,            // -(constraint_count) * 1.0
    pub constraint_program: Vec<ConstitutionalConstraint>,
    pub topology_signature: SystemTopology,
    pub evidence_chain: Vec<EvidenceEntry>,
    pub operator_versions: BTreeMap<String, String>,
    pub created_at_tick: u64,        // 0 (genesis)
    pub carrier_instance: u64,       // 0

    // Genesis-specific fields
```

```
    pub adamant_version: String,     // "1.0.0"
    pub conformance_class: ConformanceClass,
    pub system_fingerprint: SystemFingerprint,
    pub constitutional_digest: Hash256,
}

pub struct ConstitutionalConstraint {
    pub id: String,                  // "GC-01" through "GC-21"
    pub axiom_ref: String,           // ADAMANT reference
    pub description: String,         // human-readable
    pub severity: ConstraintSeverity,
    pub satisfied: bool,             // at genesis time
    pub evidence: String,            // JSON proof
}

pub enum ConformanceClass { C0, C1, C2, C3, C4 }

pub enum ConstraintSeverity { Mandatory, Recommended }

pub struct SystemFingerprint {
    pub isls_version: String,
    pub crate_count: usize,
    pub test_count: usize,
    pub registry_digest: Hash256,
    pub config_digest: Hash256,
    pub platform: String,
    pub rust_version: String,
    pub git_commit: Option<String>,
}

pub struct SystemTopology {
    pub crate_graph: BTreeMap<String, Vec<String>>,  // crate ->
       dependencies
    pub kernel_crates: Vec<String>,                  // C1-C11
    pub extension_crates: Vec<String>,               // C12+
    pub circular_dependencies: Vec<(String, String)>, // must be empty
}
```

6 Genesis Crystal Construction

Algorithm 1 BuildGenesisCrystal: Construct the system constitution

Require: Config θ, RegistrySet , system metadata
Ensure: GenesisCrystal C_0 or initialization failure

```
constraints ← LoadConstitutionalConstraints()
fingerprint ← ComputeSystemFingerprint(θ, )
topology ← ComputeSystemTopology()
// Evaluate all constraints
for each φ ∈ constraints do
    φ.satisfied ← EvaluateConstraint(φ, θ, , topology)
    φ.evidence ← GatherEvidence(φ, θ, )
end for
// Check mandatory constraints
failures ← {φ : φ.severity = Mandatory ∧ ¬φ.satisfied}
if failures ≠ ∅ then
    ReportConstitutionalFailure(failures)
    return ⊥ {initialization aborted}
end if
// Determine conformance class
class ← DetermineConformanceClass(constraints)
// Build the crystal
C_0 ← BuildCrystal(constraints, fingerprint, topology, class)
// Validate through standard 8-gate cascade
Validate8Gate(C_0) {must pass}
// Commit as first archive entry
ArchiveCommit(C_0)
return C_0
```

Requirement 6.1 (Genesis is First)**.** The Genesis Crystal **MUST** be the first entry in the archive. Its `created_at_tick` is 0. Its commit index is 0. No other crystal may have commit index 0.

Requirement 6.2 (Genesis Passes 8 Gates)**.** The Genesis Crystal itself **MUST** pass all 8 formal validation checks. It is a valid Semantic Crystal in every respect. The constitutional constraints are encoded in its `constraint_program`. Its `free_energy` is negative (computed as $-|\text{constraints}| \cdot 1.0$). Its evidence chain is complete.

Part IV
Runtime Validation

7 Genesis Validation

At every `validate formal` run, the validation harness includes a special Genesis check:

Definition 7.1 (Genesis Validation)**.** Genesis validation consists of three checks:

(GV1) **Existence**: the archive contains a crystal with `created_at_tick = 0`. If absent, the system was not properly initialized.

(GV2) **Integrity**: the Genesis Crystal passes the standard 8-gate validation (content address, evidence chain, etc.).

(GV3) **Conformance**: re-evaluate all 21 constitutional constraints against the *current* system state. If any mandatory constraint that was satisfied at genesis is now violated, report constitutional drift.

Definition 7.2 (Constitutional Drift)**.** Constitutional drift occurs when the system's current configuration no longer satisfies a mandatory constraint that the Genesis Crystal claims was satisfied at initialization. Causes include:

- Operator added to the engine but not registered in the registry (GC-02 violation).
- Gate disabled or threshold set to 0 (GC-08/GC-10 violation).
- Trace schema disabled (GC-03 violation).
- Compute budget removed (GC-16 violation).
- Shadow mode disabled without operator acknowledgment (GC-21 violation).

Requirement 7.1 (Drift Reporting)**.** When constitutional drift is detected, the system **MUST**:

(a) add a `CONSTITUTIONAL_DRIFT` alert to the alert log with severity `red`,
(b) include the drift details in every subsequent HTML report as a prominent warning banner,
(c) continue operating (the system does not self-halt), but
(d) mark every crystal produced after drift detection with `conformance_status: "drifted"` in its metadata.

The system does not enforce conformance. It makes non-conformance visible and indelible.

8 Genesis in Reports

Definition 8.1 (Report Genesis Section)**.** The HTML report (`report full-html`) includes a dedicated Genesis section:

Genesis Section in HTML Report

```
Section 0: System Constitution
  Genesis Crystal ID: <truncated hash>
  ADAMANT Version: 1.0.0
  Conformance Class: C4 (Constitutional)
  Created: 2026-03-17T04:20:00Z

  Constitutional Constraints: 21/21 satisfied

  | ID    | Axiom        | Status | Evidence          |
  |-------|-------------|--------|------------------|
  | GC-01 | Axiom 2.0.1 | PASS   | R5 normalized    |
  | GC-02 | Axiom 2.0.2 | PASS   | 47 ops registered|
  | ...   | ...         | ...    | ...              |
  | GC-21 | S18         | PASS   | shadow mode on   |

  Constitutional Drift: NONE

  System Fingerprint:
    ISLS v1.0.0 | 24 crates | 243 tests
    Registry: <digest> | Config: <digest>
    Platform: windows | Rust: 1.90.0
    Git: <commit>
```

If drift is detected:

```
  Constitutional Drift: DETECTED

  | GC-02 | Axiom 2.0.2 | DRIFT | operator "custom_op"
  |       |             |       | not in registry     |
```

Part V
Constitutional Amendment

9 Amendment Crystals

The Genesis Crystal is immutable. But systems evolve. When the constitutional commitments need to change (e.g., a new invariant is added, a conformance class is upgraded, or a constraint is relaxed with justification), the mechanism is an *Amendment Crystal*.

Definition 9.1 (Amendment Crystal)**.** An Amendment Crystal C_A is a Semantic Crystal with:

- `constraint_program`: the amended constraint set (full replacement, not delta),
- `sub_crystal_ids`: $[C_0.\text{id}]$ (references the Genesis Crystal),
- `amendment_justification`: canonical JSON explaining why each changed constraint was changed,
- `new_conformance_class`: the conformance class after amendment.

An Amendment Crystal passes the standard 8-gate validation and is committed to the archive like any other crystal. The validation harness uses the *latest amendment* (or the Genesis Crystal if no amendments exist) as the active constitution.

Requirement 9.1 (Amendment Chain)**.** Amendment Crystals form a chain: each references its predecessor (Genesis or prior amendment) via `sub_crystal_ids`. The chain is append-only. The complete constitutional history is always reconstructible from the archive.

Requirement 9.2 (No Silent Weakening)**.** If an Amendment Crystal relaxes a mandatory constraint (changes it from mandatory to recommended, or removes it), the `amendment_justification` for that constraint **MUST** be non-empty. An amendment that silently weakens a constraint without justification **MUST** fail validation.

Part VI
Implementation

10 Changes to Existing Crates

10.1 `isls-types` Extensions

```rust
// New types in isls-types/src/lib.rs

pub struct ConstitutionalConstraint {
    pub id: String,
    pub axiom_ref: String,
    pub description: String,
    pub severity: ConstraintSeverity,
```

```
    pub satisfied: bool,
    pub evidence: String,
}

pub enum ConstraintSeverity { Mandatory, Recommended }
pub enum ConformanceClass { C0, C1, C2, C3, C4 }

pub struct SystemFingerprint {
    pub isls_version: String,
    pub crate_count: usize,
    pub test_count: usize,
    pub registry_digest: Hash256,
    pub config_digest: Hash256,
    pub platform: String,
    pub rust_version: String,
    pub git_commit: Option<String>,
}

pub struct GenesisMetadata {
    pub adamant_version: String,
    pub conformance_class: ConformanceClass,
    pub system_fingerprint: SystemFingerprint,
    pub constitutional_digest: Hash256,
    pub constraints: Vec<ConstitutionalConstraint>,
}
```

10.2 isls-harness Extensions

```
// New module: isls-harness/src/genesis.rs

pub fn build_genesis_crystal(
    config: &Config,
    registries: &RegistrySet,
) -> Result<SemanticCrystal, GenesisError>;

pub fn evaluate_constitutional_constraints(
    config: &Config,
    registries: &RegistrySet,
) -> Vec<ConstitutionalConstraint>;

pub fn validate_genesis(
    archive: &Archive,
    config: &Config,
    registries: &RegistrySet,
) -> GenesisValidationResult;

pub struct GenesisValidationResult {
    pub exists: bool,
    pub integrity: bool,          // 8-gate check
    pub conformance: bool,        // all mandatory still satisfied
    pub drift: Vec<String>,       // constraint IDs that drifted
    pub conformance_class: ConformanceClass,
}

pub fn detect_constitutional_drift(
```

```
    genesis: &SemanticCrystal,
    current_config: &Config,
    current_registries: &RegistrySet,
) -> Vec<DriftEntry>;

pub struct DriftEntry {
    pub constraint_id: String,
    pub was: bool,                // satisfied at genesis
    pub now: bool,                // satisfied now
    pub detail: String,
}
```

10.3 isls-cli Extensions

```
// Enhanced init command
isls init [--store sqlite] [--profile standard]
// Now creates Genesis Crystal as first archive entry

// New genesis commands
isls genesis show          // Display Genesis Crystal summary
isls genesis validate      // Run GV1-GV3 checks
isls genesis amend --justification amend.json
                           // Create an Amendment Crystal

// Enhanced validate command
isls validate formal       // Now includes Genesis validation
isls validate genesis      // Genesis-specific validation

// Enhanced report
isls report full-html      // Now includes Section 0: Constitution
```

10.4 isls-store Extension

```
// New table for constitutional history

CREATE TABLE constitution (
    crystal_id    TEXT PRIMARY KEY,
    is_genesis    BOOLEAN NOT NULL,
    is_amendment  BOOLEAN NOT NULL,
    conformance   TEXT NOT NULL,     -- 'C0'..'C4'
    constraints   TEXT NOT NULL,     -- JSON array
    created_at    TEXT NOT NULL
);

// Query: get active constitution
// (latest amendment, or genesis if no amendments)
pub fn get_active_constitution(&self) -> Result<ConstitutionRow>;
```

Part VII
Acceptance Tests

ID	Name	Procedure
AT-G1	Genesis creation	Run `isls init`; verify archive contains exactly one crystal with tick=0.
AT-G2	Genesis 8-gate	Validate the Genesis Crystal; verify all 8 checks pass.
AT-G3	Genesis constraints	Inspect Genesis Crystal; verify 21 constraints present, all mandatory ones satisfied.
AT-G4	Conformance class	With default config, verify conformance class is C4.
AT-G5	Constitutional drift	After genesis, modify config to disable trace; run `validate genesis`; verify GC-03 drift detected. Restore config; verify drift clears.
AT-G6	Missing genesis	Delete genesis crystal from archive; run `validate formal`; verify GV1 failure reported.
AT-G7	Amendment crystal	Create amendment that adds a constraint; verify amendment references genesis; verify new constraint checked on subsequent validation.
AT-G8	No silent weakening	Create amendment that removes a mandatory constraint without justification; verify amendment fails validation.
AT-G9	Genesis in report	Generate HTML report; verify Section 0 present with constraint table and fingerprint.
AT-G10	Double init prevention	Run `isls init` on an already-initialized system; verify error "genesis already exists."

Part VIII
Handoff

11 Test Count

Phase	New Tests	Cumulative
Phase 1–5.1	231	231
Genesis Crystal	10	$\geq$ 241

12 Verification Sequence

```
# Build
cargo build --workspace --release 2>&1 | grep -c warning  -> 0

# Test
cargo test --workspace -- --test-threads=1  -> >= 241 passed

# Initialize with Genesis Crystal
rm -rf ~/.isls
cargo run --bin isls --release -- init

# Verify Genesis exists and is valid
cargo run --bin isls --release -- genesis show
cargo run --bin isls --release -- genesis validate
# -> Genesis Crystal: VALID, Conformance: C4, 21/21 constraints

# Run a scenario
cargo run --bin isls --release -- ingest --adapter synthetic --scenario
   S-Basic
cargo run --bin isls --release -- run --mode shadow --ticks 100

# Validate includes Genesis check
cargo run --bin isls --release -- validate formal
# -> Genesis: VALID | Crystals: 50/50 pass

# Generate report with Section 0
cargo run --bin isls --release -- report full-html
# -> Section 0: System Constitution present

# Test drift detection
# (temporarily modify a config value that violates GC-XX)
cargo run --bin isls --release -- genesis validate
# -> DRIFT DETECTED: GC-XX
```

13 Completion Criteria

The Genesis Crystal specification is complete when:

1. `isls init` creates a Genesis Crystal as the first archive entry.
2. The Genesis Crystal passes all 8 formal validation checks.
3. All 21 constitutional constraints are evaluated and encoded.
4. Conformance class is correctly determined.
5. Constitutional drift is detected when config changes.
6. Amendment Crystals work and reference the Genesis.
7. HTML report includes Section 0: System Constitution.
8. All 10 acceptance tests pass.
9. Total tests ≥ 241, zero warnings.

Generated for ISLS Genesis Crystal v1.0.0.
The constitution is not a document. It is a crystal.
It is not read. It is checked.

Deterministic, append-only, replay-verified.
ADAMANT.